Backend · revision notes

Django

MVT, ORM and migrations, views, DRF APIs, auth, admin, security

1 What Django is: the MVT flow

Definition

Django is a "batteries-included" Python web framework: ORM, migrations, admin, auth, forms, templates and security defaults in one package. It follows MVT: Model, View, Template.

Think of Django like a restaurant

1



Customer

Browser request

Orders a URL, e.g. /menu/

2



Waiter reads the order

urls.py + View

Routes the request to the right function

3



Kitchen + pantry

Model / ORM

Fetches and prepares the data from the database

4



Plated dish

Template / JSON

The response sent back to the customer

Request → Middleware → urls.py (URL router) → View →
Model / ORM ↔ Database → Template or JSON → Middleware → Response

Start a project

```
python -m venv .venv && source .venv/bin/activate
pip install django
django-admin startproject shop .
python manage.py startapp products
python manage.py migrate
python manage.py createsuperuser
python manage.py runserver
```

- A **project** is the whole site (settings, root URLs); an **app** is one feature (products, users) you can reuse.
- Register apps in `INSTALLED_APPS`; configuration lives in `settings.py` (secrets and DEBUG come from environment variables).
- Django is synchronous by default; ASGI support allows async views and WebSockets (Channels).

2 Models, ORM and migrations

Think of the ORM like a translator at a meeting

1



You speak Python

`Product.objects.filter(...)`

Readable code

2



Translator

Django ORM

Turns it into SQL

3



Database speaks SQL

PostgreSQL

Runs the query

4



Answer translated back

Model objects

Rows come back as Python objects

products/models.py

```python
from django.db import models

class Category(models.Model):
    name = models.CharField(max_length=80, unique=True)

class Product(models.Model):
    category = models.ForeignKey(Category, on_delete=models.CASCADE,
related_name="products")
    name = models.CharField(max_length=120)
    price = models.DecimalField(max_digits=8, decimal_places=2)
    in_stock = models.BooleanField(default=True)
    created = models.DateTimeField(auto_now_add=True)

    class Meta:
        ordering = ["-created"]
        indexes = [models.Index(fields=["name"])]

    def __str__(self):
        return self.name
```

Querying with the ORM

```python
from django.db.models import Count, Q

Product.objects.filter(in_stock=True, price__lt=500).order_by("name")
Product.objects.exclude(category__name="Toys").first()
Product.objects.filter(Q(name__icontains="pen") | Q(price__gt=1000))

# N+1 fix: join in one query instead of one query per row
for p in Product.objects.select_related("category"):        # ForeignKey /
OneToOne
    print(p.name, p.category.name)
cats = Category.objects.prefetch_related("products")         # reverse FK /
ManyToMany

Category.objects.annotate(n=Count("products")).filter(n__gt=5)
```

- After changing models: `makemigrations` (writes the migration file) then `migrate` (applies it). Commit migration files.
- QuerySets are lazy: nothing hits the database until you iterate, slice or call `list()`, `.count()` or `.exists()`.

- `select_related` (SQL JOIN) for single-valued relations and `prefetch_related` (extra query) for many-valued ones fix the **N+1 problem**.
- Wrap multi-step writes in `transaction.atomic()`. Use `F()` expressions for race-free updates (`F("stock") - 1`).

Common mistake

Looping over products and touching product.category inside the loop runs one extra query per row. Add select_related("category").

3 URLs, views, forms and templates

Think of a view like a receptionist

1



Visitor asks for /products/

URL

urls.py picks the receptionist

2



Receptionist

View

Gets data from models

3



Fills a letter template

Template

Inserts the data into HTML

4



Sends the letter back

HttpResponse

Browser shows the page

A URL, a view and a template

```python
# products/urls.py
from django.urls import path
from . import views
urlpatterns = [
    path("", views.product_list, name="product-list"),
    path("<int:pk>/", views.ProductDetail.as_view(), name="product-detail"),
]

# products/views.py
from django.shortcuts import get_object_or_404, render
from django.views.generic import DetailView
from .models import Product

def product_list(request):                          # function-based view
    products = Product.objects.select_related("category")
    return render(request, "products/list.html", {"products": products})

class ProductDetail(DetailView):                    # class-based view
    model = Product
```

products/list.html (template language)

```
{% extends "base.html" %}
{% block content %}
  {% for p in products %}
    <h2>{{ p.name }} - ₹{{ p.price }}</h2>      <!-- auto-escaped against XSS
-->
  {% empty %}
    <p>No products.</p>
  {% endfor %}
{% endblock %}
```

- **FBV** (function-based views) are simple and explicit; **CBV** (class-based) reuse behaviour through generic views like `ListView`, `DetailView`, `CreateView`.
- **Forms / ModelForms** validate input and render fields; `{% csrf_token %}` is required in POST forms.
- **Middleware** wraps every request (security, sessions, auth, CSRF, your own logging).
- Use `reverse("product-detail", args=[1])` and `{% url %}` instead of hard-coding URLs.

4 Building APIs with Django REST framework

Think of DRF like a translator between JSON and objects

1



Client sends JSON

Request

Text over the network

2



Check and convert

Serializer

Validates and builds a Python object

3



Save to the database

ViewSet / Model

Create, read, update, delete

4



Send JSON back

Response

Serializer converts the object to JSON

Serializer + ViewSet + router

```python
from rest_framework import serializers, viewsets, permissions
from rest_framework.routers import DefaultRouter

class ProductSerializer(serializers.ModelSerializer):
    class Meta:
        model = Product
        fields = ["id", "name", "price", "category"]

    def validate_price(self, value):
        if value <= 0:
            raise serializers.ValidationError("Price must be positive")
        return value

class ProductViewSet(viewsets.ModelViewSet):     # list, create, retrieve,
update, delete
    queryset = Product.objects.select_related("category")
    serializer_class = ProductSerializer
    permission_classes = [permissions.IsAuthenticatedOrReadOnly]

router = DefaultRouter()
router.register("products", ProductViewSet)
urlpatterns = router.urls
```

- **Serializers** convert model instances to JSON and validate incoming data (like forms for APIs).
- **ViewSets + routers** generate the standard REST routes; add `pagination_class`, filters and throttling in settings.
- Authentication options: session, token, **JWT** (SimpleJWT). Permissions decide *who may do what*.
- Return the right status codes (201, 400, 401, 403, 404) and keep one consistent error shape.

5 Auth, admin, security and deployment

Think of Django security like a hotel front desk

1



Guest shows ID

Login

Django checks hashed passwords

2



Room key card

Session / token

Proves who you are on each request

3



Only your floor

Permissions

login_required, groups, roles

4



Cameras and locks

CSRF, XSS, SQL-injection protection

Built in by default

- Built-in `User` model, login/logout views, password hashing (PBKDF2/Argon2), groups and permissions. Start with a **custom user model** (`AUTH_USER_MODEL`) in new projects.
- `@login_required` / `LoginRequiredMixin` protect views; `@permission_required` checks specific permissions.
- The **admin** (`admin.site.register(Product)`) gives you instant CRUD for staff: customise with `ModelAdmin` (`list_display`, `search_fields`).
- Built-in protections: ORM parameterises queries (SQL injection), templates auto-escape (XSS), CSRF middleware, clickjacking header, secure cookies.
- Production: `DEBUG=False`, set `ALLOWED_HOSTS`, keep `SECRET_KEY` in env vars, `collectstatic` + WhiteNoise/CDN, run with **gunicorn/uvicorn** behind Nginx, use PostgreSQL.
- Background work (emails, reports) goes to **Celery** + Redis, not inside the request.
- Test with `django.test.TestCase` or pytest-django; run `python manage.py check --deploy` before launch.

Revision

MVT flow, startproject vs startapp, models → makemigrations → migrate, lazy QuerySets, select_related/prefetch_related for N+1, FBV vs CBV, DRF serializers + ViewSets, CSRF/XSS/SQLi protections, DEBUG off, gunicorn behind Nginx, Celery for background jobs.

6 Rapid-fire Q&A

Q. What is the MVT pattern in Django?

A. Model (data via the ORM), View (logic that handles a request and returns a response), Template (HTML presentation). The URL router picks the view.

Q. Project vs app in Django?

A. A project is the whole website with settings and root URLs; an app is a reusable module that implements one feature.

Q. What are migrations?

A. Version-controlled files that describe changes to your models; makemigrations creates them and migrate applies them to the database.

Q. What is the N+1 problem and how does Django fix it?

A. One query for the list plus one extra query per row for a relation. Use select_related (JOIN) or prefetch_related (batched query).

Q. select_related vs prefetch_related?

A. select_related does a SQL JOIN for ForeignKey/OneToOne; prefetch_related runs a separate query and joins in Python, for ManyToMany and reverse relations.

Q. Are QuerySets lazy?

A. Yes. The database is only hit when you evaluate the QuerySet (iterate, slice with a step, list(), len(), bool, .count()/.exists()).

Q. Function-based vs class-based views?

A. FBVs are explicit functions; CBVs use classes and mixins to reuse common behaviour such as list/detail/create pages.

Q. What is middleware?

A. A hook that processes every request and response (security headers, sessions, auth, CSRF), configured in the MIDDLEWARE list in order.

Q. How does Django prevent CSRF?

A. The CsrfViewMiddleware requires a secret token with unsafe methods (POST/PUT/DELETE); forms include {% csrf_token %}.

Q. What is DRF and what is a serializer?

A. Django REST framework adds API tools. A serializer converts model instances to JSON and validates incoming data.

Q. How do you deploy Django?

A. DEBUG off, ALLOWED_HOSTS and SECRET_KEY set, collectstatic, run gunicorn/uvicorn behind Nginx, PostgreSQL, and Celery for background tasks.

Q. What do signals do?

A. They let decoupled code react to events such as post_save of a model; use sparingly because they hide control flow.